

DESIGN AND IMPLEMENTATION OF A CENTRALIZED LOG MONITORING AND THREAT DETECTION SYSTEM USING ELK STACK

Submitted By

Shubham Mahato

INTRODUCTION

Modern IT environments generate large volumes of event logs from operating systems, applications, and network services. These logs contain valuable information regarding user activities, system events, errors, and potential security incidents. Monitoring logs individually across multiple systems can be difficult and time-consuming. Centralized log monitoring addresses this challenge by collecting logs from different sources and storing them in a single location for analysis.

This project implements a centralized log monitoring and threat detection pipeline using the ELK Stack together with Winlogbeat. Windows Event Logs are collected from a Windows virtual machine and forwarded to a centralized Ubuntu-based ELK server. The collected logs are processed, indexed, and stored for monitoring and investigation.

OBJECTIVES

- Centralize Windows event logs
- Process and forward logs using Logstash
- Store and index logs in Elasticsearch
- Verify successful log ingestion through Winlogbeat indices
- Demonstrate a basic SOC-style workflow

ELK STACK COMPONENTS

Elasticsearch: Stores and indexes log data.

Logstash: Receives and processes logs before forwarding them.

Winlogbeat: Collects Windows Event Logs and ships them to Logstash.

SYSTEM ARCHITECTURE

Windows VM → Winlogbeat → Logstash → Elasticsearch

SCREENSHOTS AND EVIDENCE

Screenshot 1: Elasticsearch Running Successfully

```
C:\Users\Student\Desktop\winlogbeat-9.4.1-windows-x86_64\winlogbeat-9.4.1-windows-x86_64>sc query winlogbeat

SERVICE_NAME: winlogbeat
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE                : 4   RUNNING
                                (STOPPABLE, NOT_PAUSABLE, ACCEPTS_SHUTDOWN)
        WIN32_EXIT_CODE       : 0   (0x0)
        SERVICE_EXIT_CODE   : 0   (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT            : 0x0
```

Screenshot 2: Logstash Listening on Port 5044

```
shubham@ubuntu-ELK:~$ ss -tulp | grep 5044

tcp    LISTEN 0      4096      *:5044      *:*
```

SCREENSHOTS AND EVIDENCE

Screenshot 3: Winlogbeat Service Running

```
shubham@Ubuntu-ELK:~$ curl http://localhost:9200
{
  "name" : "Ubuntu-ELK",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "1aBum5b8TauRKwLeZ6yF-g",
  "version" : {
    "number" : "8.19.15",
    "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "d9256c374e649e04ff0fa2dafd43402d35a3fb3a",
    "build_date" : "2026-04-28T13:06:49.648073236Z",
    "build_snapshot" : false,
    "lucene_version" : "9.12.2",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

Screenshot 4: Winlogbeat Indices Created in Elasticsearch

```
shubham@Ubuntu-ELK:~$ curl http://localhost:9200/_cat/indices?v
health status index                                uuid                                pri rep docs.count
green open .internal.alerts-default.alerts-default-000001 HdrPKCs0QeKBexHaNNM9Xw 1 0
0 0 249b 249b 249b
yellow open winlogbeat-2026.05.28 -nDoAW-WQAKfxc448DubXA 1 1 1
81 0 742.9kb 742.9kb 742.9kb
yellow open winlogbeat-2026.05.30 wU1sbnj8SmiMNTupeNtkDA 1 1 4
78 0 940.6kb 940.6kb 940.6kb
yellow open winlogbeat-2026.05.27 6c3SGH2_RAGHS8xipvGnyw 1 1
35 0 286.1kb 286.1kb 286.1kb
yellow open winlogbeat-2026.05.26 U063PN6qRBqK4_fbqQLDiQ 1 1 3
29 0 586.5kb 586.5kb 586.5kb
shubham@Ubuntu-ELK:~$ _
```

CONCLUSION

The project successfully demonstrated the implementation of a centralized log monitoring and threat detection system using the ELK Stack. Windows Event Logs were collected through Winlogbeat, transmitted to Logstash, and stored within Elasticsearch. The successful creation of Winlogbeat indices confirmed that log ingestion and centralized storage were functioning correctly.

The completed solution provides a practical understanding of how centralized logging is implemented in modern environments. By collecting and organizing logs in a single platform, administrators can efficiently monitor system activity and investigate potential incidents.

LIMITATIONS

Due to hardware resource constraints of the testing environment, the complete deployment of Kibana for continuous visualization was not maintained during the final verification phase. Available system memory was prioritized for Elasticsearch and Logstash services to ensure stable log collection, processing, and storage. Despite this limitation, all primary objectives of the project were successfully achieved.

FUTURE SCOPE

The system can be enhanced by integrating Kibana dashboards, automated alerting mechanisms, additional log sources, and advanced threat detection rules. These improvements can transform the solution into a more comprehensive SIEM platform suitable for larger-scale monitoring environments.